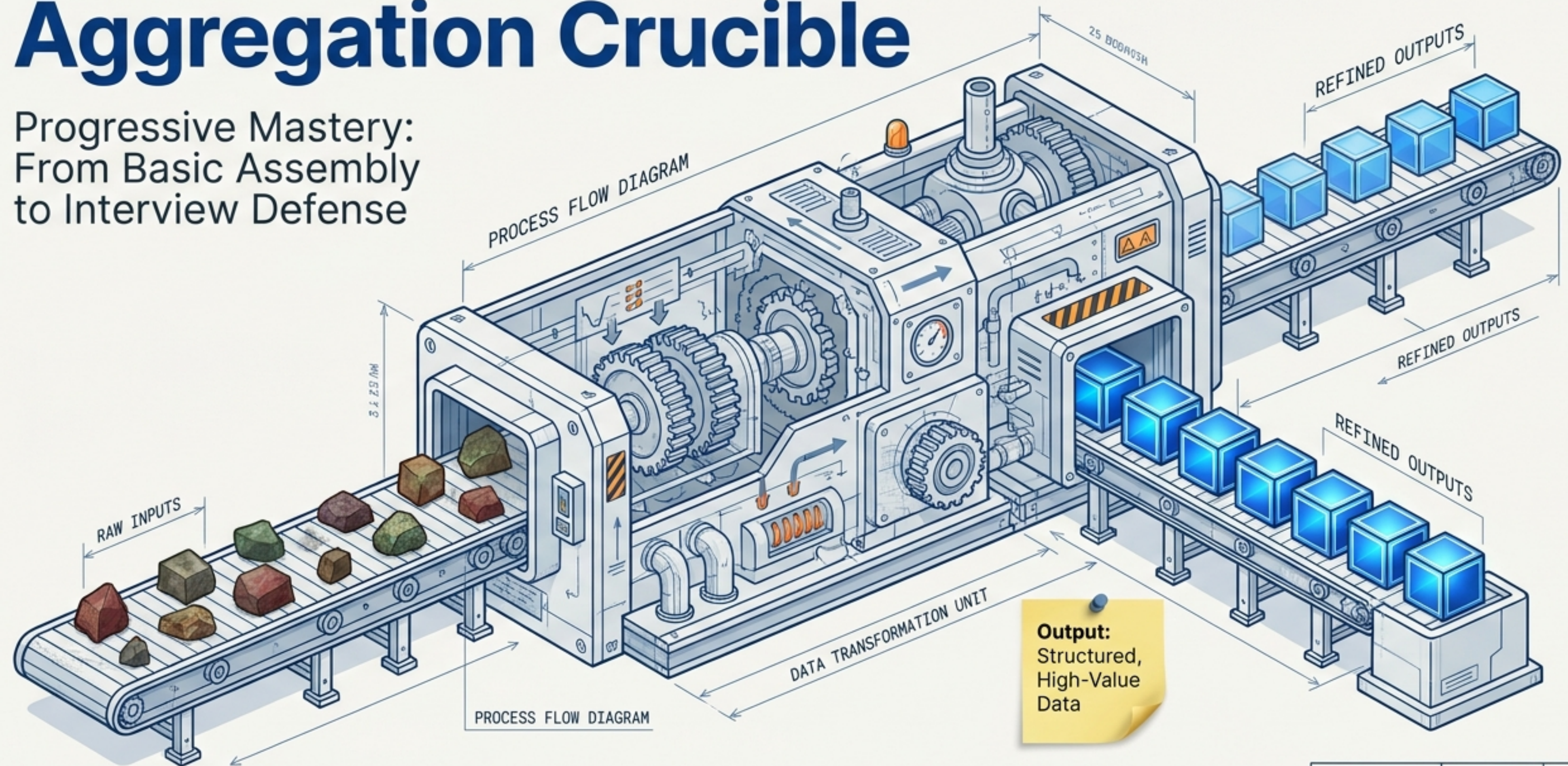


The MongoDB Aggregation Crucible

Progressive Mastery:
From Basic Assembly
to Interview Defense



The Mental Model: The Data Assembly Line

Pipeline is a factory assembly line.
Filter before you assemble.



STRUCTURED
REPORT DOCUMENT

```
{  
  "group_id": "A",  
  "count": 45,  
  "average_value": 120.5  
}
```

Station A (Filter)

Station B (Reshape)

Station C (Group)

Reshape & Simplify

\$project

\$match

\$group

REJECT BIN
(FILTERED OUT)

Data Aggregation
& Categorization

The aggregation pipeline processes documents through discrete stages.
Each stage transforms the data and passes it to the next.

Station 1: Filter & Flow Control

```
{  
  $match: { name: 'A' }  
}
```

Same as a find() query. Filters early.

```
{  
  $sort: { name: 1, age: -1 }  
}
```

1 for ascending, -1 for descending.

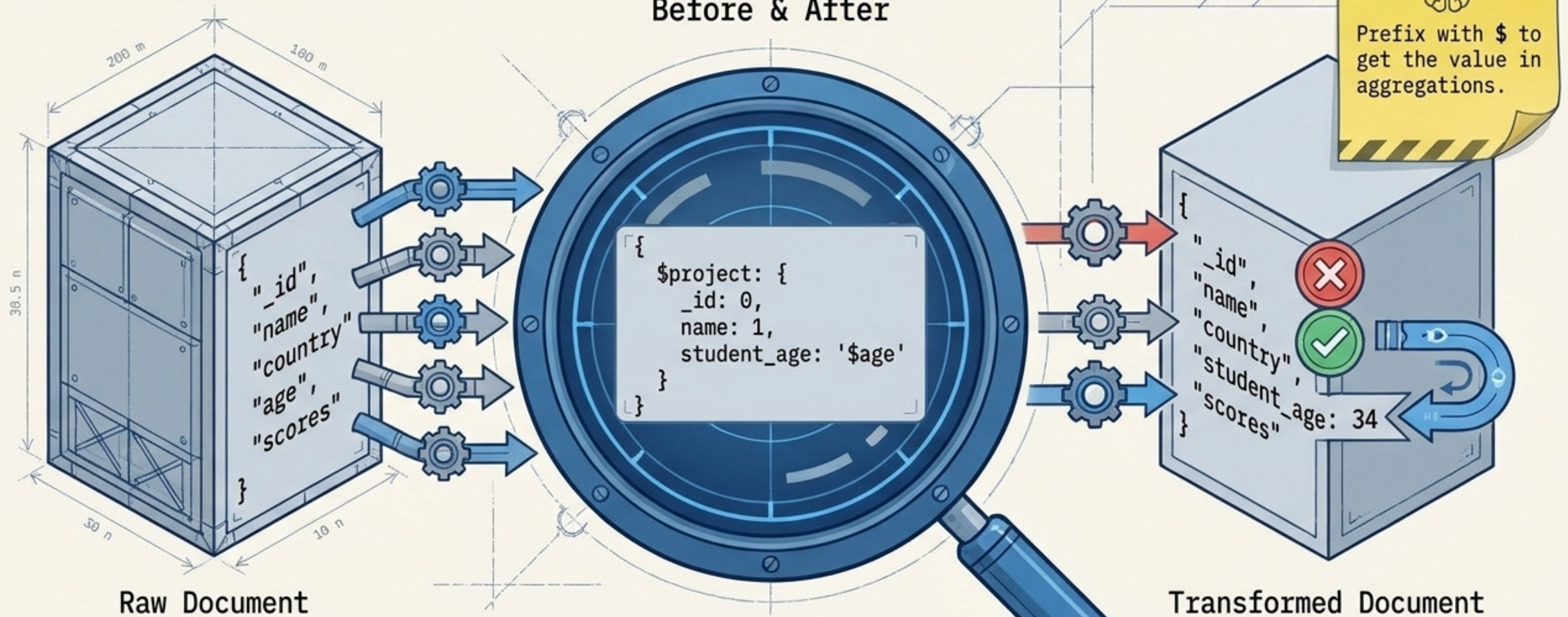
```
{ $skip: 2 }
```

```
{ $limit: 4 }
```

Always place \$match as early as possible in your pipeline.

Station 2: Reshaping the Payload (\$project & \$addFields)

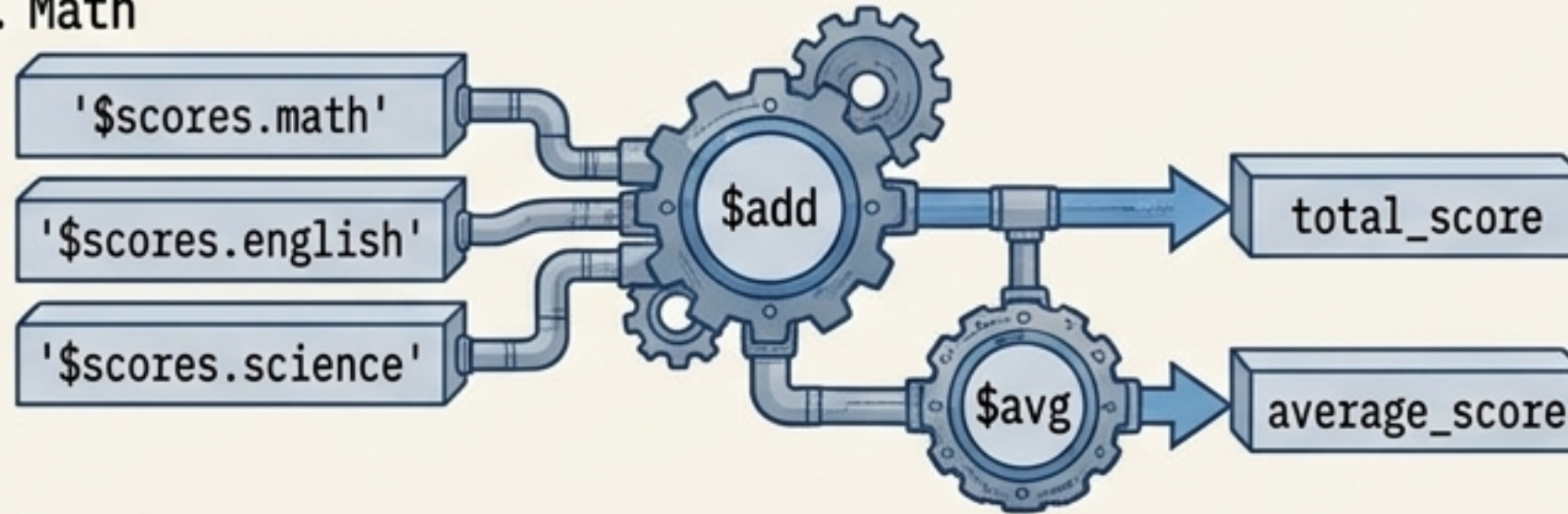
Before & After



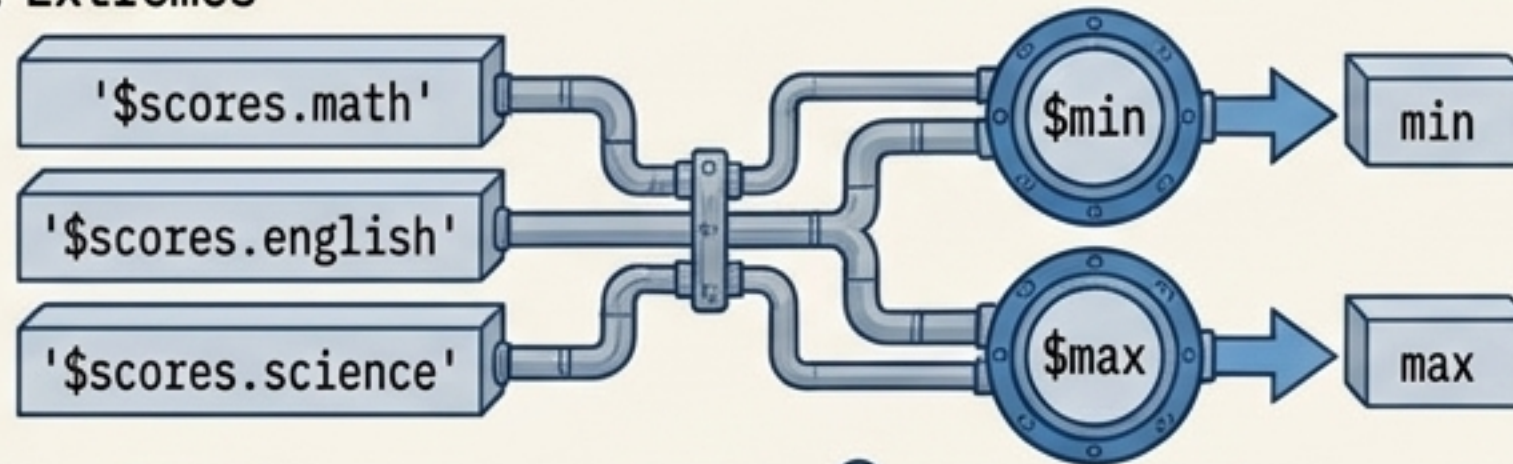
Use `$project` to reshape the output. If you just want to append new fields without dropping others, use `$addFields`.

Station 3: The Computation Engine

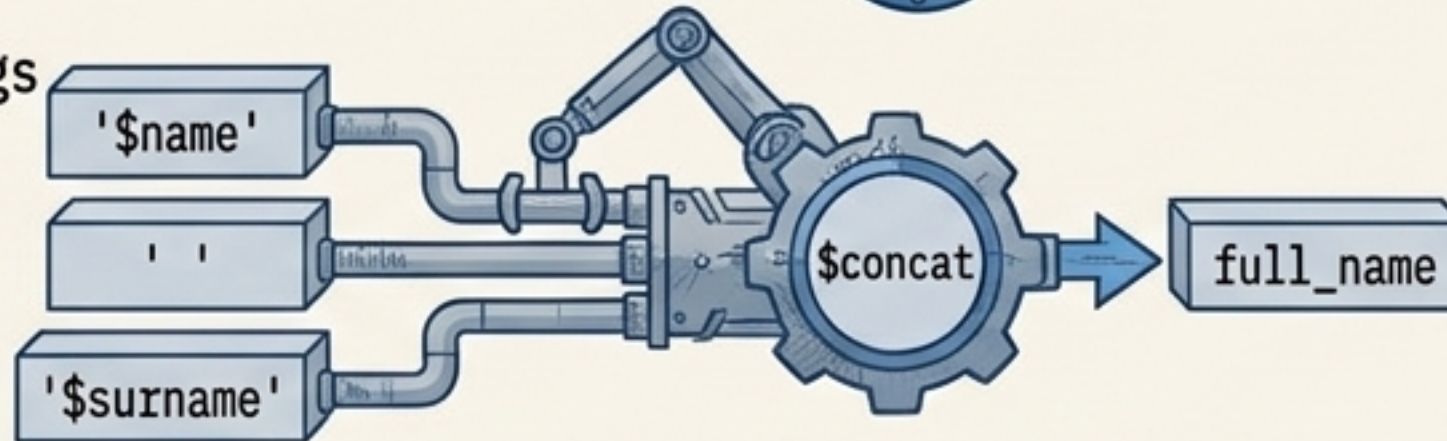
A. Math



B. Extremes



C. Strings

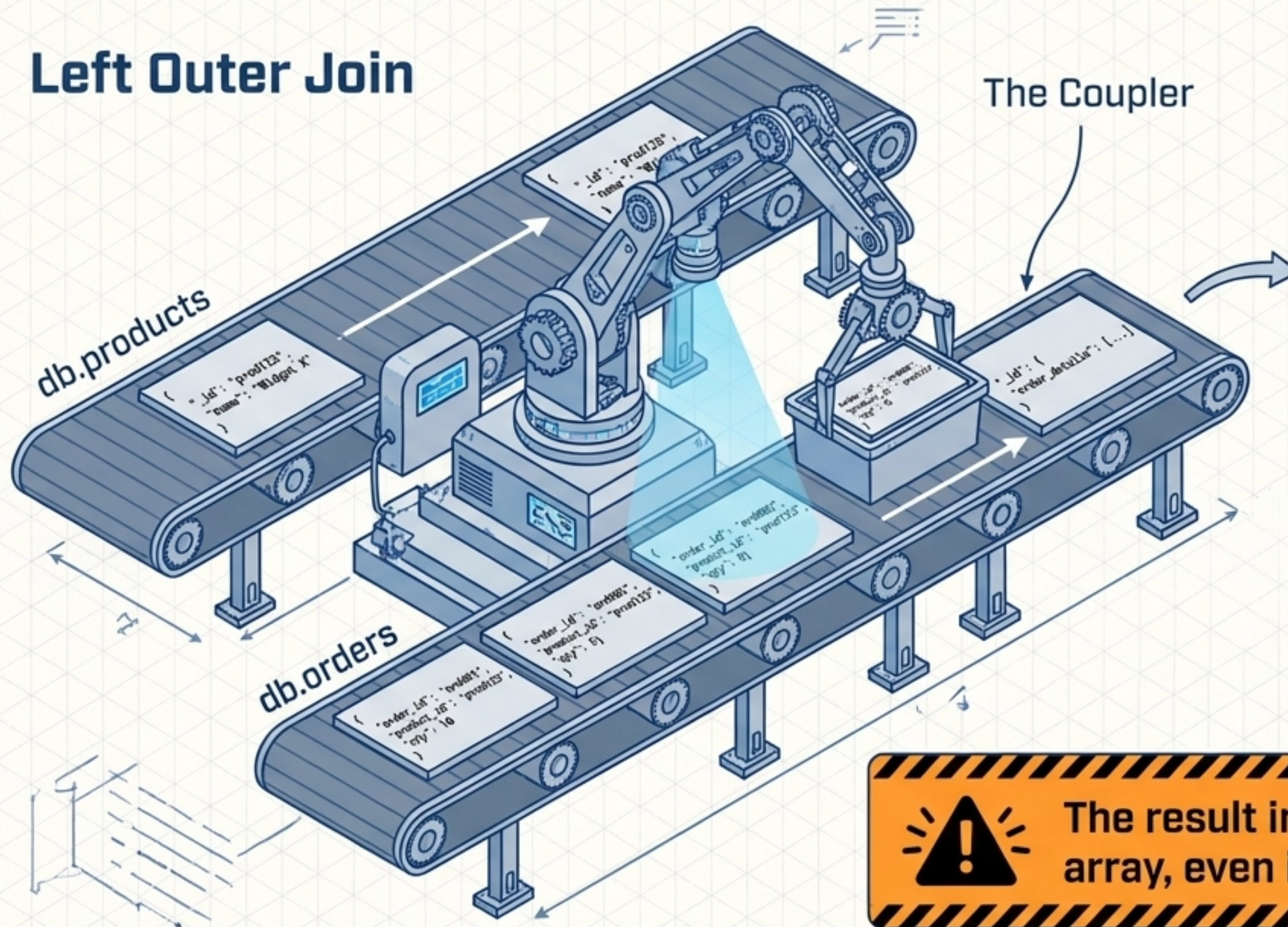


```
percentage: {
  $multiply: [
    { $divide: [
      { $add: [
        '$scores.math',
        '$scores.english',
        '$scores.science'
      ] },
      300
    ] },
    100
  ]
}
```

Field names in arithmetic expressions MUST be prefixed with \$ (e.g., '\$scores.math'), otherwise MongoDB reads them as literal strings!

Station 4: The Relational Coupler (\$lookup)

Left Outer Join



`$lookup`
=
`SQL JOIN`

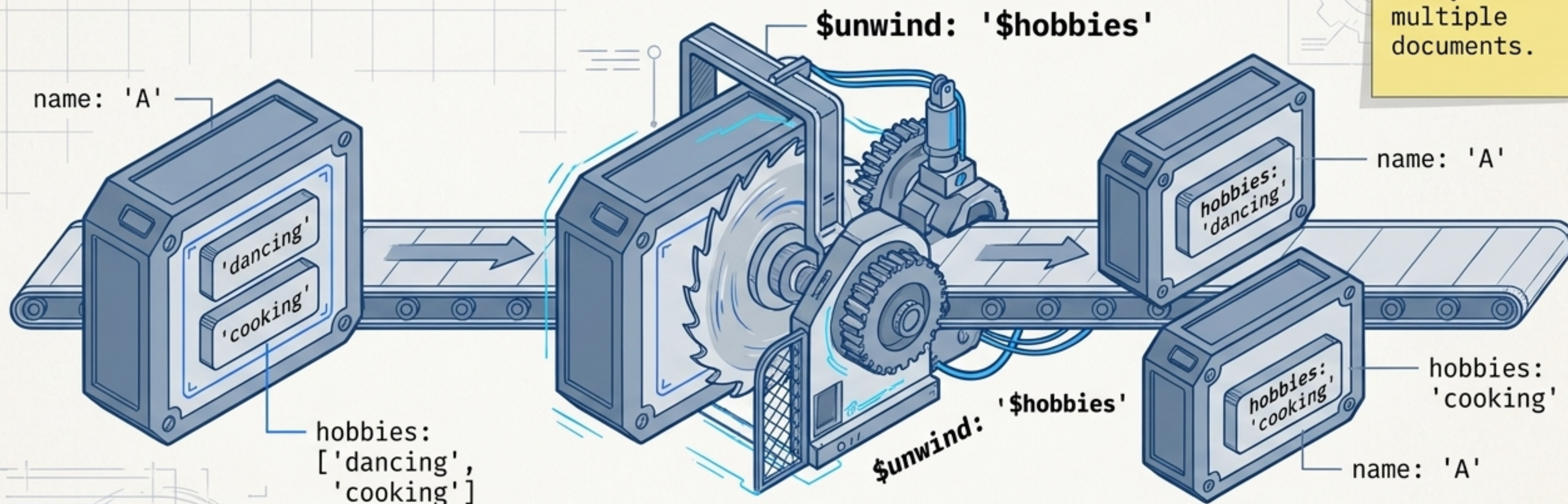
```
{ $lookup: {  
  from: 'orders',  
  localField: '_id',  
  foreignField: 'product_id',  
  as: 'order_details'  
}}
```



The result inside the 'as' field is ALWAYS an array, even if there is only one match!

Station 5: The Array Exploder (\$unwind)

\$unwind explodes an array into multiple documents.

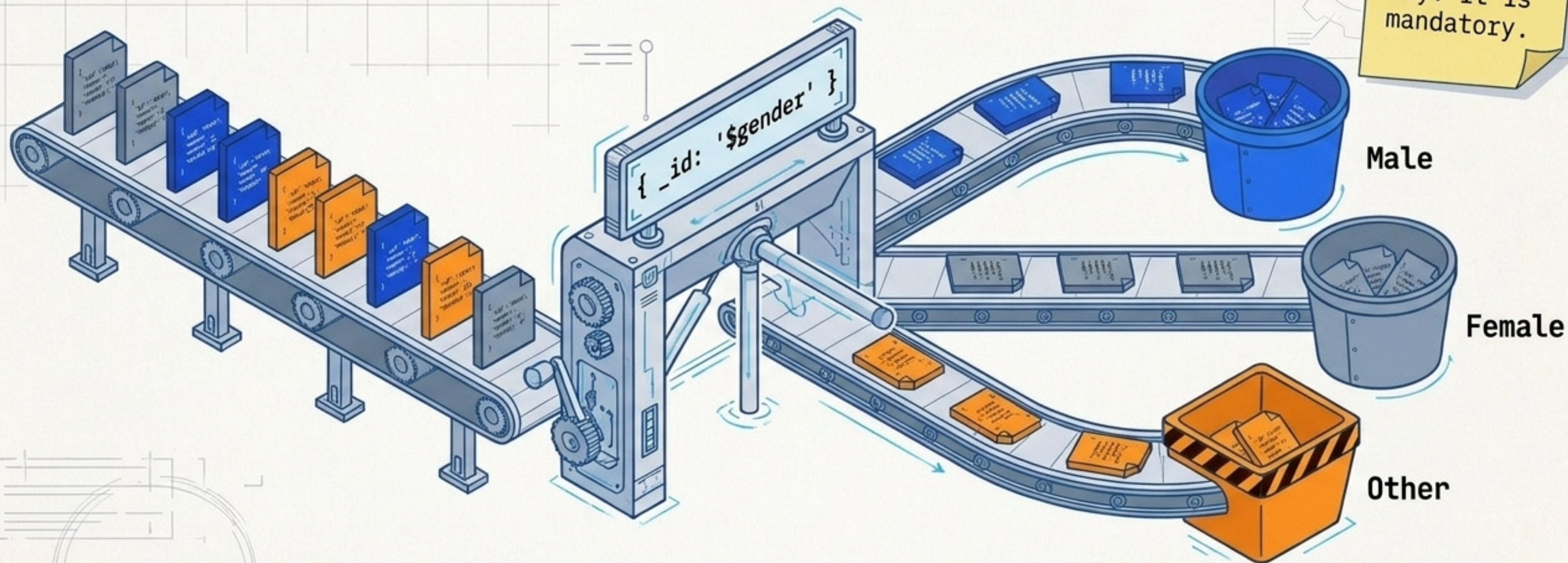


If a document has an empty array or **null** for that field, **\$unwind** removes the document entirely!



Safety Net: Always use `{ preserveNullAndEmptyArrays: true }` if you don't want to lose them.

Station 6: The Aggregation Hub (\$group)



Pro-Tip: How do you find a global metric (like average price of ALL products)? Set the **routing key** to `null: { _id: null }`. All data flows into a single master bucket.

The Accumulator Matrix (Inside \$group)

Array Builders

\$push: 'The Hoarder' - Keeps everything.

```
[ { category: { $push: '$category' } } ]
```



\$addToSet: 'The Curator' - Keeps only unique values.

```
[ { brand: { $addToSet: '$brand' } } ]
```



Math/Stats

\$sum: Adds values (use **\$sum: 1** to count total documents)

\$avg: Calculates the mean (**\$avg: '\$age'**)

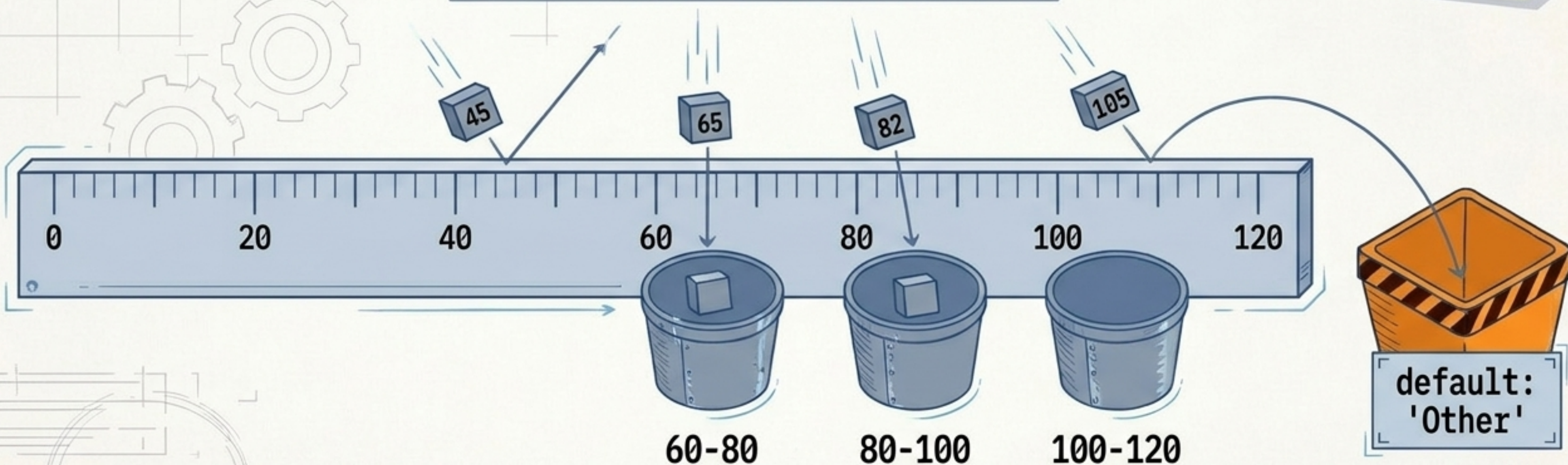
\$max / \$min: Extracts the absolute highest/lowest values in the grouping.

Accumulators require \$group (or \$bucket) to function. You cannot use them in \$match.

Station 7: The Histogram Generator (\$bucket)

\$bucket =
Histogram
grouping

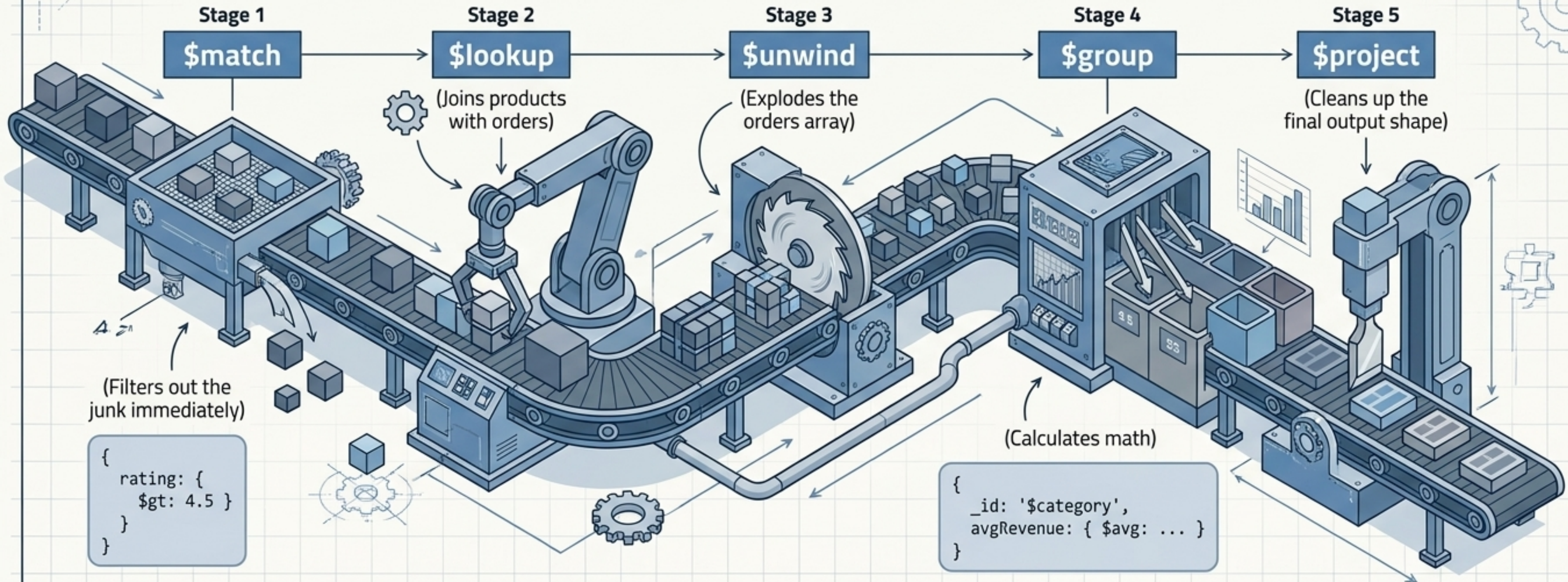
```
{ boundaries: [60, 80, 100] }
```



Boundaries must be sorted. Lower bound is inclusive, upper bound is exclusive.
You **MUST** provide a 'default' bucket for outliers, or MongoDB will throw an error.

Architectural Synthesis: The Master Pipeline

Goal: Find the average revenue of highly-rated products by category.



The mastery of MongoDB lies not in knowing individual commands, but in composing this symphony of data transformation.

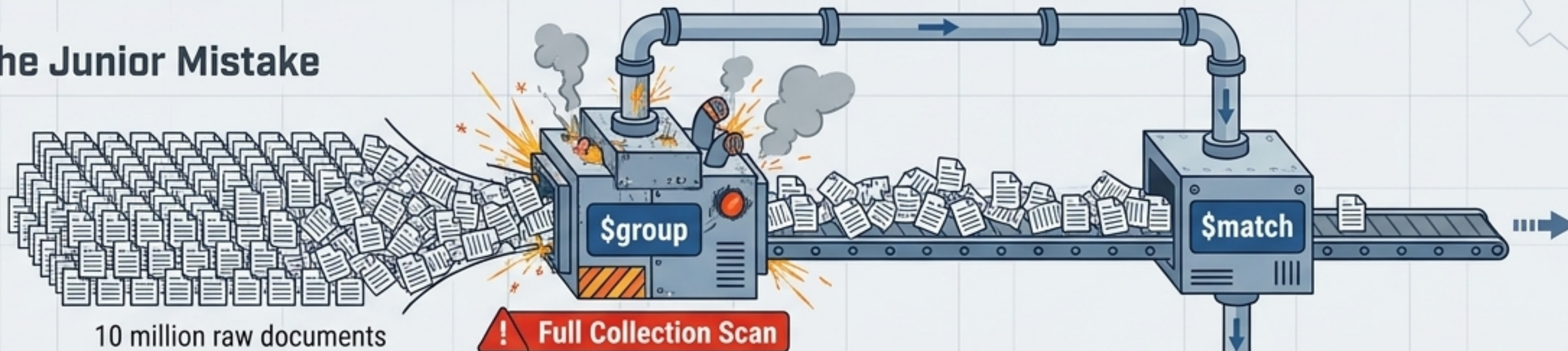
Viva Prep: The 'Gotcha' Diagnostic Table

HAZARD (COMMON PITFALLS)		SOLUTION (CORRECTIVE ACTIONS)	
	\$lookup outputs arrays, making downstream math impossible.		Immediately follow \$lookup with \$unwind .
	\$unwind silently deletes documents with empty/null arrays.		Use { preserveNullAndEmptyArrays: true }.
	\$bucket throws a fatal error if a document falls outside boundaries.		Always define a 'default' catch-all bucket.
	Projection arithmetic returns literal strings instead of values.		Ensure fields are prefixed with \$ (e.g., '\$price').

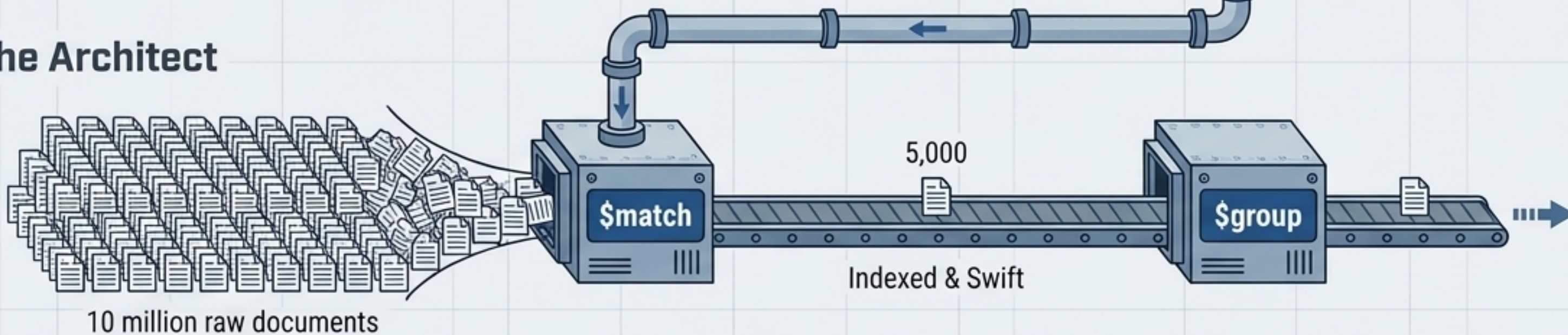
The Grilling: Order of Operations & Performance



The Junior Mistake

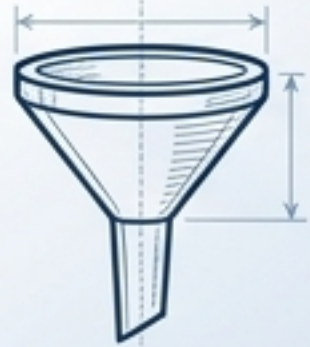


The Architect



Filtering with **\$match** first drastically reduces the dataset size in memory and utilizes indexes. Grouping or projecting first destroys index utility.

The Cheat Sheet: Rapid-Recall Mnemonics



\$match

Filter before
you assemble.



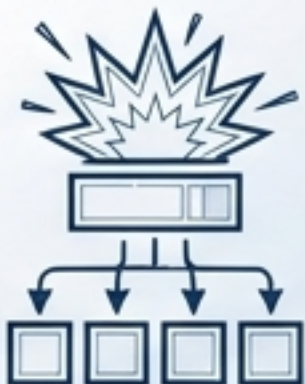
\$project

Prefix with \$
to get values.



\$lookup

It's a SQL JOIN.
Output is
always an array.



\$unwind

Explodes an
array into
multiple docs.



\$group

_id is the
mandatory
'GROUP BY' key.



\$bucket

The Histogram
Generator.

The Examiner's Crucible (Simulation)

Why is the order of stages important in an aggregation pipeline?

Filtering with **\$match** first reduces dataset size and leverages indexes. Grouping first forces a **full collection scan**. ⚠️

Think: Filter before you assemble.

How do you group all documents together to find a global average?

Set the **_id** field in the **\$group** stage to **null**:

```
{ $group: { _id: null, avg: { $avg: '$price' } } }.
```

Cheat:
_id: null means 'Global Group'

What data type does the 'as' field in a **\$lookup** produce?

It **ALWAYS** produces an **array** containing all matching documents, even if there is only a single match.

Recall:
It's a SQL JOIN. Output is always an array.

You are ready. Defend your pipeline.